03. Slow Path 컨센서스 동작과정

목차

1. Slow Path 개념
2. Bullshark 통합
3. 비잔틴 행동 감지
4. 자동 폴백 메커니즘
5. 상태 보존 및 복원
6. 고급 비잔틴 패턴 감지

Slow Path 개념

Slow Path는 비잔틴 장애 허용을 위한 안전한 폴백 메커니즘으로, Narwhal의 검증된 Bullshark 알고리즘을 래핑하여 구현됩니다. Fast Path에서 문제가 감지되거나 네트워크 조건이 악화될 때 자동으로 활성화됩니다.

Slow Path 초기화

```
impl SlowPathConsensus {
pub fn new(committee: Committee, fallback_threshold: Stake) -> Self {
// 테스트용 저장소 생성 (실제로는 외부에서 제공)
let store = Self::create_test_store();
let bullshark = Bullshark::new(committee.clone(), store, 50); // gc_depth = 50
```

```
// 제네시스 인증서로 초기 컨센서스 상태 생성
let genesis = Certificate::genesis(&committee);
let consensus_state = ConsensusState::new(genesis, Arc::new(Default::default())


Self {
committee,
bullshark: Arc::new(RwLock::new(bullshark)),
fallback_threshold,
consensus_state: Arc::new(RwLock::new(consensus_state)),
byzantine_detected: Arc::new(RwLock::new(false)),
}
}
}
```

외부 저장소와 함께 생성

```
impl SlowPathConsensus {
pub fn new_with_store(
committee: Committee,
store: Arc<ConsensusStore>,
fallback_threshold: Stake,
gc_depth: Round,
) -> Self {
```

```
let bullshark = Bullshark::new(committee.clone(), store, gc_depth);
let genesis = Certificate::genesis(&committee);
let consensus_state = ConsensusState::new(genesis, Arc::new(Default::default())


Self {
committee,
bullshark: Arc::new(RwLock::new(bullshark)),
fallback_threshold,
consensus_state: Arc::new(RwLock::new(consensus_state)),
byzantine_detected: Arc::new(RwLock::new(false)),
}
}
}
```

Bullshark 통합

인증서 처리

```
impl SlowPathConsensus {
pub async fn process_certificate(&mut self, certificate: &Certificate) -> Re
let mut bullshark = self.bullshark.write().await;
let mut state = self.consensus_state.write().await;
// 플레이스홀더 컨센서스 인덱스 (실제로는 적절히 관리됨)
```

```
let consensus_index = 0;
match bullshark.process_certificate(&mut *state, consensus_index, certificate.c
Ok(_outputs) => {
// 인증서 처리 성공
Ok(())
}
Err(e) => {
// 컨센서스 오류 처리
Err(AutobahnError::NarwhalConsensusError(format!("Bullshark error: {:?}", e)))
}
}
}
}
```

위원회 업데이트

```
impl SlowPathConsensus {
pub async fn update_committee(&mut self, new_committee: Committee) -> Result
self.committee = new_committee.clone();
let mut bullshark = self.bullshark.write().await;
match bullshark.update_committee(new_committee) {
Ok(()) => Ok(()),
Err(e) => Err(AutobahnError::NarwhalConsensusError(
```

```
format!("Committee update failed: {:?}", e)
)),
}
}
}
```

비잔틴 행동 감지

종합적인 비잔틴 감지

```
impl SlowPathConsensus {
pub  async  fn  detect_byzantine_behavior(&mut  self, certificate: &Certificate)
let  round = certificate.round();
let  author = certificate.origin();
let  state_guard = self.consensus_state.read().await;
// 1. 이중 투표 감지 (Equivocation)
if  let Some(round_certificates) = state_guard.dag.get(&round) {
if  let Some((existing_digest, existing_cert)) = round_certificates.get(&author)
if  existing_cert.digest() != certificate.digest() {
tracing::error!("Equivocation detected from author {:?} in round {}", author, ro
self.set_byzantine_detected(true).await;
return Ok(true);
}
```

```
}
}
// 2. 잘못된 부모 참조 감지
if  self.detect_invalid_parents(certificate, &state_guard).await? {
tracing::error!("Invalid parent references detected in certificate from {:?}", a
self.set_byzantine_detected(true).await;
return Ok(true);
}
// 3. 라운드 진행 위반 감지
if  self.detect_round_violations(certificate, &state_guard).await? {
tracing::error!("Round progression violation detected from {:?}", author);
self.set_byzantine_detected(true).await;
return Ok(true);
}
// 4. 스테이크 임계값 위반 감지
if  self.detect_stake_violations(certificate).await? {
tracing::error!("Stake threshold violation detected in certificate from {:?}", a
self.set_byzantine_detected(true).await;
return Ok(true);
}
Ok(false)
}
```

```
}
```

잘못된 부모 참조 감지

```
impl SlowPathConsensus {
async fn detect_invalid_parents(&self, certificate: &Certificate, state: &Cons
let current_round = certificate.round();
// 부모는 이전 라운드에서 와야 함
let expected_parent_round = current_round.saturating_sub(1);
for parent_digest in &certificate.header.parents {
let mut parent_found = false;
if let Some(parent_round_certs) = state.dag.get(&expected_parent_round) {
for (_, (digest, _)) in parent_round_certs {
if digest == parent_digest {
parent_found = true;
break;
}
}
}
if !parent_found {
// 예상 라운드에서 부모를 찾을 수 없음 - 잠재적 비잔틴 행동
return Ok(true);
}
```

```
}
Ok(false)
}
}
```

라운드 진행 위반 감지

```
impl SlowPathConsensus {
async fn detect_round_violations(&self, certificate: &Certificate, state: &Con
let author = certificate.origin();
let current_round = certificate.round();
// 작성자가 이미 더 높은 라운드에 커밋했는지 확인
if let Some(&last_committed_round) = state.last_committed.get(&author) {
if current_round <= last_committed_round {
// 작성자가 이미 지나간 라운드에 인증서를 제출하려고 함
return Ok(true);
}
}
// 합리적인 라운드 진행 확인 (너무 앞서가지 않음)
let max_allowed_round = state.last_committed_round + 10; // 어느 정도 유연성 허용
if current_round > max_allowed_round {
return Ok(true);
```

```
}
Ok(false)
}
}
```

스테이크 임계값 위반 감지

```
impl SlowPathConsensus {
async fn detect_stake_violations(&self, certificate: &Certificate) -> Result<b
// 인증서가 적절한 스테이크 지원을 받는지 확인
let author_stake = self.committee.stake(&certificate.origin());
let min_required_stake = self.committee.validity_threshold();
// 단일 작성자가 너무 많은 스테이크를 가지는지 확인 (잠재적 중앙화 공격)
if author_stake > min_required_stake {
tracing::warn!("Single author has excessive stake: {} > {}", author_stake, min_r
// 이것이 비잔틴 행동은 아닐 수 있지만 모니터링할 가치가 있음
}
Ok(false)
}
}
```

자동 폴백 메커니즘

폴백 조건 확인

```
impl SlowPathConsensus {
pub  async  fn  should_fallback_to_slow_path(&self, byzantine_stake: Stake) -> b
// 비잔틴 스테이크가 임계값을 초과하거나 비잔틴 행동이 감지된 경우 폴백
byzantine_stake >= self.fallback_threshold || self.is_byzantine_detected().await
}
}
```

자동 폴백 트리거

```
impl SlowPathConsensus {
pub  async  fn  trigger_automatic_fallback(&mut  self, reason: TransitionReason)
tracing::info!("Triggering automatic fallback to slow path, reason: {:?}", reaso
match  reason {
TransitionReason::ByzantineDetected => {
self.set_byzantine_detected(true).await;
self.handle_byzantine_fallback().await?;
}
TransitionReason::FastPathTimeout => {
self.handle_timeout_fallback().await?;
}
TransitionReason::NetworkDegradation => {
```

```
self.handle_network_fallback().await?;
}
_ => {
tracing::warn!("Unexpected fallback reason: {:?}", reason);
}
}
Ok(())
}
}
```

폴백 유형별 처리

비잔틴 폴백 처리

```
impl SlowPathConsensus {
async fn handle_byzantine_fallback(&mut self) -> Result<(), AutobahnError> {
// 보안 조치 강화
// 실제로는 다음을 포함할 수 있음:
// 1. 더 보수적인 컨센서스 매개변수로 전환
// 2. 검증 요구사항 증가
// 3. 추가 모니터링 활성화
tracing::warn!("Activated Byzantine fallback mode - enhanced security measures e
Ok(())
```

```
}

}
```

타임아웃 폴백 처리

```
impl SlowPathConsensus {

async fn handle_timeout_fallback(&mut self) -> Result<(), AutobahnError> {

// Slow Path용 타임아웃 매개변수 조정

// 실제로는 다음을 포함할 수 있음:

// 1. 타임아웃 값 증가

// 2. 재시도 메커니즘 조정

// 3. 네트워크 매개변수 수정

tracing::info!("Activated timeout fallback mode - adjusted network parameters");

Ok(())

}

}
```

네트워크 성능 저하 폴백 처리

```
impl SlowPathConsensus {

async fn handle_network_fallback(&mut self) -> Result<(), AutobahnError> {

// 열악한 네트워크 조건에 맞게 조정

// 실제로는 다음을 포함할 수 있음:

// 1. 메시지 빈도 감소
```

```
// 2. 버퍼 크기 증가
// 3. 압축 활성화
tracing::info!("Activated network degradation fallback mode - optimized for poor
Ok(())
}
}
```

상태 보존 및 복원

상태 스냅샷 생성

```
pub struct StateSnapshot {
pub consensus_state: ConsensusState,
pub byzantine_detected: bool,
pub committee: Committee,
pub fallback_threshold: Stake,
pub last_preserved_round: Round,
}

impl SlowPathConsensus {
pub async fn preserve_consensus_state(&self) -> Result<StateSnapshot, Autobah
let state_guard = self.consensus_state.read().await;
let byzantine_detected = self.is_byzantine_detected().await;
```

```
Ok(StateSnapshot {
consensus_state: state_guard.clone(),
byzantine_detected,
committee: self.committee.clone(),
fallback_threshold: self.fallback_threshold,
last_preserved_round: state_guard.last_committed_round,
})
}
}
```

상태 복원

```
impl SlowPathConsensus {
pub  async  fn  restore_consensus_state(&mut  self, snapshot: StateSnapshot) ->
// 복원 전 스냅샷 유효성 검증
if  snapshot.committee.epoch() != self.committee.epoch() {
return Err(AutobahnError::ModeTransitionFailed {
from: ConsensusMode::FastPath,
to: ConsensusMode::SlowPath,
reason: "Committee epoch mismatch during state restoration".to_string(),
});
}
```

```
// 컨센서스 상태 복원
{
let mut state_guard = self.consensus_state.write().await;
*state_guard = snapshot.consensus_state;
}

// 비잔틴 감지 상태 복원
self.set_byzantine_detected(snapshot.byzantine_detected).await;

// 필요시 위원회 업데이트
if snapshot.committee != self.committee {
self.update_committee(snapshot.committee).await?;
}

tracing::info!("Successfully restored consensus state from round {}", snapshot.l
Ok(())
}
}
```

안전한 전환 실행

```
impl SlowPathConsensus {
```

```
pub  async  fn  execute_safe_transition(
&mut  self,
target_mode: ConsensusMode,
reason: TransitionReason
) -> Result<StateSnapshot, AutobahnError> {
// 전환이 안전한지 확인
if !self.is_transition_safe(target_mode).await? {
return Err(AutobahnError::ModeTransitionFailed {
from: ConsensusMode::SlowPath,
to: target_mode,
reason: "Unsafe to transition - pending operations detected".to_string(),
});
}


// 전환 전 현재 상태 보존
let  snapshot = self.preserve_consensus_state().await?;


// 전환별 로직 실행
match  target_mode {
ConsensusMode::FastPath => {
self.prepare_for_fast_path_transition().await?;
}
ConsensusMode::SlowPath => {
```

```
self.trigger_automatic_fallback(reason).await?;
}
ConsensusMode::Transitioning => {
self.enter_transitioning_state().await?;
}
}


tracing::info!("Successfully executed transition to {:?}", target_mode);
Ok(snapshot)
}
}
```

고급 비잔틴 패턴 감지

비잔틴 패턴 유형

```
pub  enum ByzantinePattern {
/// 이중 투표: 같은 라운드에 다른 인증서 전송
Equivocation {
author: types::PublicKey,
round: Round,
certificates: Vec<types::CertificateDigest>,
},
```

```
/// 조정된 공격: 여러 노드가 협력하여 행동
CoordinatedAttack {
participants: Vec<types::PublicKey>,
attack_type: String,
evidence: Vec<u8>,
},
/// 타이밍 공격: 인증서 타이밍 조작
TimingAttack {
author: types::PublicKey,
suspicious_timing: Vec<(Round, u64)>,
},
/// 보류 공격: 의도적으로 인증서 보류
WithholdingAttack {
author: types::PublicKey,
missing_rounds: Vec<Round>,
},
}
```

고급 패턴 감지

```
impl SlowPathConsensus {
pub  async  fn  detect_advanced_byzantine_patterns(&mut  self) -> Result<Vec<Byz
let  state_guard = self.consensus_state.read().await;
```

```
let mut detected_patterns = Vec::new();
// 1. 조정된 공격 감지
if let Some(pattern) = self.detect_coordinated_attack(&state_guard).await? {
detected_patterns.push(pattern);
}
// 2. 타이밍 공격 감지
if let Some(pattern) = self.detect_timing_attack(&state_guard).await? {
detected_patterns.push(pattern);
}
// 3. 보류 공격 감지
if let Some(pattern) = self.detect_withholding_attack(&state_guard).await? {
detected_patterns.push(pattern);
}
if !detected_patterns.is_empty() {
self.set_byzantine_detected(true).await;
}
Ok(detected_patterns)
}
}
```

비잔틴 노드 응답 처리

```
impl SlowPathConsensus {
```

```
pub  async  fn  handle_byzantine_response(&mut  self, byzantine_node: &types::Pu

// 비잔틴 행동 감지로 표시

self.set_byzantine_detected(true).await;

// 포렌식을 위한 비잔틴 행동 로깅

tracing::warn!("Byzantine behavior detected from node: {:?}", byzantine_node);

// 실제로는 다음을 수행할 수 있음:

// 1. 향후 컨센서스에서 비잔틴 노드 제외

// 2. 모니터링 시스템에 보고

// 3. 추가 보안 조치 트리거

Ok(())

}

}
```

SlowPathOperations 트레이트 구현

```
#[async_trait]

impl SlowPathOperations for SlowPathConsensus {

async  fn  process_slow_path_certificate(&mut  self, certificate: &Certificate)

// 먼저 비잔틴 행동 감지

let  byzantine_detected = self.detect_byzantine_behavior(certificate).await?;

if  byzantine_detected {

tracing::warn!("Byzantine behavior detected while processing certificate: {:?}",
```

```
}
// Bullshark를 사용하여 인증서 처리
self.process_certificate(certificate).await
}
async fn should_use_slow_path(&self) -> bool {
// 비잔틴 행동이 감지된 경우 항상 Slow Path 사용
self.is_byzantine_detected().await
}
async fn handle_fast_path_fallback(&mut self, reason: TransitionReason) -> Re
match reason {
TransitionReason::ByzantineDetected => {
self.set_byzantine_detected(true).await;
tracing::info!("Switched to slow path due to Byzantine behavior detection");
}
TransitionReason::FastPathTimeout => {
tracing::info!("Switched to slow path due to fast path timeout");
}
TransitionReason::NetworkDegradation => {
tracing::info!("Switched to slow path due to network degradation");
}
_ => {
tracing::info!("Switched to slow path due to: {:?}", reason);
}
```

```
}

Ok(())

}

}
```

Slow Path 처리 흐름 요약

1. **초기화**: Bullshark 래핑 및 컨센서스 상태 설정
2. **인증서 처리**: 비잔틴 감지 후 Bullshark로 처리
3. **비잔틴 감지**: 다양한 패턴으로 악의적 행동 탐지
4. **자동 폴백**: Fast Path에서 문제 발생 시 자동 전환
5. **상태 관리**: 모드 전환 시 안전한 상태 보존/복원
6. **고급 감지**: 복잡한 비잔틴 공격 패턴 식별

Slow Path는 Autobahn BFT의 안전망 역할을 하며, 어떤 상황에서도 비잔틴 장애 허용을 보장합니다.